Business Requirements

rooster

Problem

Open-source CLI tools that allow one to launch, monitor, and store locally-based single-cluster GPU runs are very limited. Existing options, such as WandB and Tensorboard, only provide surface-level customizations through a user-friendly terminal interface. A lightweight, fast, and customizable option is key to providing each user with the freedom to monitor all training activities from the terminal.

Solution

An open source tool, 'rrooster', that allows one to configure, visualize, and monitor local single-node GPU cluster training. It gives the user the freedom to define any sort of metric from the training run(loss, perplexity, etc), and allows a visually informal & rich TUI design in the terminal. With write and read safeties using on-device SQLite, it guarantees logged metrics during unexpected failures.

Nouns and Verbs:

- User wants **local, on-device** training monitoring
- User wants to **launch** a single-node GPU training run from the CLI
- User wants to **define custom metrics** (loss, perplexity, etc.) per run
- User wants to **visualize** live training data in a rich TUI
- User wants to **monitor** active GPU training sessions from the terminal
- User wants a **lightweight and fast** tooling with no cloud dependency
- System needs to **write metrics safely** to SQLite during a run
- System needs to **read metrics back** for display and history
- The system needs to **recover logged data** after an unexpected failure
- System needs to **store run history** locally on-device via SQLite
- User wants **full configurability** over what is tracked and how it is displayed

Design

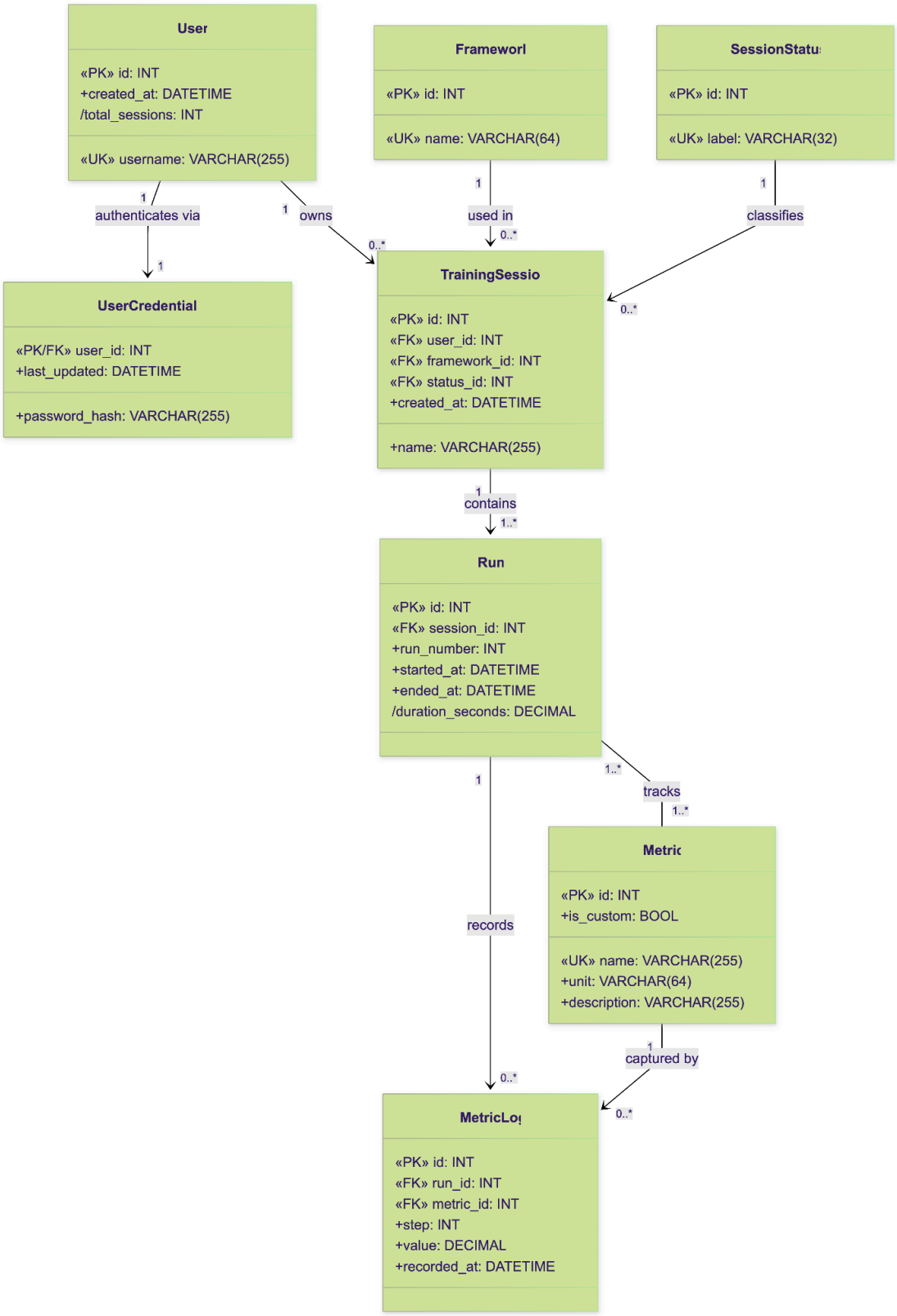
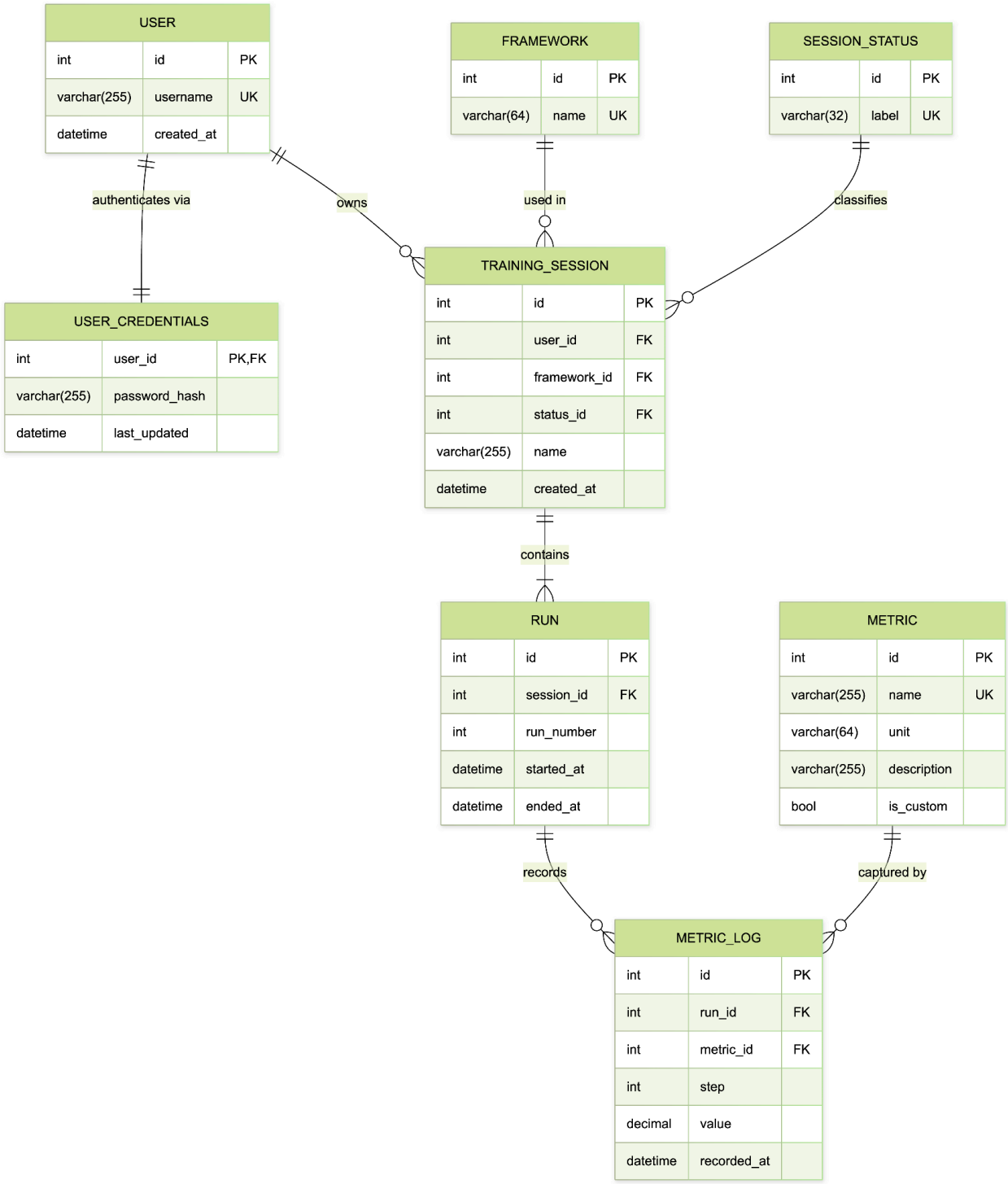
On-top of the user's training script in either Jax or Pytorch, Rooster is wrapped around the forward run pass, where in each step, it sends the user-requested metrics through a web socket to the backend receiver on the same device. With it, the data is deserialized and then formatted into a database. With each metric, there is a different table representing the data being collected. For example, if the user is collecting loss function across the run, the lossFunction table would include attributes such as "loss", "time", "step #", etc. The user can decide essentially what each table should track.

Each training session can have several runs, so each run has its own smaller tables representing metrics. Using Rust's sqlx crate, it writes a table for each training session and saves its respective metric tables for each forward pass. This is the writing side of the application. Simultaneously, the CLI tool reads the training metrics being written as they are being written freshly to the database, and queries it up in a visual interface using Rataui. With the speed of read/write from the db, the readings are near instant as the forward steps are being recorded and sent from the client side.

Crows Foot:

USER(<u>id</u>, username, created_at)
USER_CREDENTIALS(<u>user_id</u>, password_hash, last_updated)
FRAMEWORK(<u>id</u>, name)
SESSION_STATUS(<u>id</u>, label)
TRAINING_SESSION(<u>id</u>, *user_id*, *framework_id*, *status_id*, name, created_at)
RUN(<u>id</u>, *session_id*, run_number, started_at, ended_at)
METRIC(<u>id</u>, name, unit, description, is_custom)
METRIC_LOG(<u>id</u>, *run_id*, *metric_id*, step, value, recorded_at)

UML Diagram


User
«PK» id: INT
+created_at: DATETIME
/total_sessions: INT
«UK» username: VARCHAR(255)
Frameworl
«PK» id: INT
«UK» name: VARCHAR(64)
SessionStatu
«PK» id: INT
«UK» label: VARCHAR(32)
1
authenticates via
1
1
owns
0..*
1
used in
0..*
1
classifies
0..*
UserCredential
«PK/FK» user_id: INT
+last_updated: DATETIME
+password_hash: VARCHAR(255)
TrainingSessio
«PK» id: INT
«FK» user_id: INT
«FK» framework_id: INT
«FK» status_id: INT
+created_at: DATETIME
+name: VARCHAR(255)
1
contains
1..*
Run
«PK» id: INT
«FK» session_id: INT
+run_number: INT
+started_at: DATETIME
+ended_at: DATETIME
/duration_seconds: DECIMAL
1
records
0..*
1..*
tracks
1..*
Metric
«PK» id: INT
+is_custom: BOOL
«UK» name: VARCHAR(255)
+unit: VARCHAR(64)
+description: VARCHAR(255)
1
captured by
0..*
MetricLo
«PK» id: INT
«FK» run_id: INT
«FK» metric_id: INT
+step: INT
+value: DECIMAL
+recorded_at: DATETIME

ERD Crows Diagram